

Introduction to ora1R

Cristina de-la-Rosa-Gay and Daniel Fernández

- 1. Overview
- 2. Dental notation tools
 - 2.1. Detecting notation
 - 2.2. Handling ambiguous notation
 - 2.3. Parsing notation
- 3. Converting dental notation
- 4. Data structuring: `tidy_dental()`
- 5. Caries indices: DMFT and def
 - 5.1. Normalization
 - 5.2. Computing DMFT
 - 5.3. Computing def
- 6. Periodontal indices
 - 6.1. Input validation: `validate_index_input()`
 - 6.2. Bleeding on Probing (BOP): `compute_bop()`
 - 6.3. Plaque Control Record (PCR): `compute_pcr()`
 - 6.4. Plaque Index (PI): `compute_pi()`
 - 6.5. Periodontal Screening Index (PSI): `compute_psi()`
- 7. Orthodontic index: ICON
 - 7.1. Component scoring: `score_icon_components()`
 - 7.2. Global ICON scoring: `compute_icon()`
- 8. Statistical tools
 - 8.1. Descriptive statistics: `analyze_descriptives()`
 - 8.2. Prevalence: `analyze_prevalence()`
- 9. Complete example workflow (9 synthetic patients).
 - 9.1. Detecting notation
 - 9.2. Converting all tooth codes to FDI notation
 - 9.3. Computing DMFT per patient
 - 9.4. Computing def per patient
 - 9.5. Computing BOP
 - 9.6. Computing PCR
 - 9.7. Computing PI
 - 9.8. Computing PSI
 - 9.9. Statistical tools
- 10. Integration with statistical workflows
- 11. Conclusions

1. Overview

ora1R is an R package designed to standardize, validate and analyse dental data. It provides tools for:

- detecting and interpreting dental notation systems
- converting between FDI, Universal and Alphanumeric
- transforming heterogeneous datasets into tidy formats
- computing some dental indices (DMFT, def, BOP, PCR, PI, PSI, ICON)
- generating descriptive statistics and prevalence estimates
- integrating results with graphical tools such as *ggplot2*

This vignette introduces the main workflow of the package.

2. Dental notation tools

2.1. Detecting notation

The package includes functions to detect, interpret and convert dental codes.

`detect_notation()` and `detect_notation_single()` identify which notation systems (FDI, Universal, Alphanumeric) are compatible with each input value.

These functions ensure that any dental code can be interpreted consistently.

Example:

```
detect_notation(c("11", "URB", "A", "55"))
```

```
## [[1]]
## [1] "Universal" "FDI"
##
## [[2]]
## [1] "Alphanumeric"
##
## [[3]]
## [1] "Universal"
##
## [[4]]
## [1] "FDI"
```

2.2. Handling ambiguous notation

Some numeric codes (11–18, 21–28, 31, 32) may correspond to both FDI and Universal. The function `handle_ambiguous_notation()` allows the user to decide how to treat them.

Example:

```
handle_ambiguous_notation(c("11", "12", "21"), action = "as_fdi")
```

```
## [1] "11" "12" "21"
## attr(,"ambiguous_as")
## [1] "FDI"
```

2.3. Parsing notation

`parse_notation()` analyses a vector of dental codes and returns a structured table based on the following logic:

- unambiguous codes: returns the specific notation system and anatomical components (quadrant and tooth type: permanent or primary).
- ambiguous codes: if a code can be FDI or Universal, it is labeled ambiguous and the function

returns NA for quadrant and type to prevent data interpretation errors.

Example:

```
parse_notation(c("11", "46", "URB", "A", "30"))
```

```
## # A tibble: 5 × 4
##   input notation    quadrant type
##   <chr> <chr>      <dbl> <chr>
## 1 11    ambiguous      NA <NA>
## 2 46    FDI           4 permanent
## 3 URB   Alphanumeric    1 primary
## 4 A     Universal      1 primary
## 5 30    Universal      4 permanent
```

The following example illustrates a complete workflow in which dental codes are first inspected to determine their possible notation systems, then ambiguous numeric codes are resolved according to a user-defined preference, and finally all codes are parsed into standardized quadrant and tooth-type components.

Example:

```
# Example dataset: all codes in Universal notation, including ambiguous ones
df <- tibble(
  tooth = c("1", "11", "24", "12", "31", "32", "A", "J")
)
```

```
# Step 1 – Detect notation (ambiguous numeric codes will be flagged)
df$detected <- detect_notation(df$tooth)
df
```

```
## # A tibble: 8 × 2
##   tooth detected
##   <chr> <list>
## 1 1     <chr [1]>
## 2 11    <chr [2]>
## 3 24    <chr [2]>
## 4 12    <chr [2]>
## 5 31    <chr [2]>
## 6 32    <chr [2]>
## 7 A     <chr [1]>
## 8 J     <chr [1]>
```

```
# Step 2 – Resolve ambiguous numeric codes as Universal
resolved <- handle_ambiguous_notation(df$tooth, action = "as_universal")
resolved
```

```
## [1] "1" "11" "24" "12" "31" "32" "A" "J"
## attr("ambiguous_as")
## [1] "Universal"
```

```

attr(resolved, "ambiguous_as")

## [1] "Universal"

# Step 3 – Parse the resolved Universal codes
parsed <- parse_notation(resolved)
parsed

## # A tibble: 8 × 4
##   input notation quadrant type
##   <chr> <chr>         <dbl> <chr>
## 1 1      Universal      1 permanent
## 2 11     Universal      2 permanent
## 3 24     Universal      3 permanent
## 4 12     Universal      2 permanent
## 5 31     Universal      4 permanent
## 6 32     Universal      4 permanent
## 7 A      Universal      1 primary
## 8 J      Universal      2 primary

```

3. Converting dental notation

The package includes conversion functions between FDI, Universal and Alphanumeric. Conversions are safe and symmetric.

Example:

```

# Step 4 – Convert the resolved Universal notation to FDI
fdi <- convert_universal_to_fdi(resolved)
fdi

## [1] "18" "23" "31" "24" "47" "48" "55" "65"
## attr(,"ambiguous_as")
## [1] "FDI"

# Step 5 – Parse the resulting FDI codes
parsed_fdi <- parse_notation(fdi)
parsed_fdi

## # A tibble: 5 × 4
##   input notation quadrant type
##   <chr> <chr>         <dbl> <chr>
## 1 18     FDI          1 permanent
## 2 23     FDI          2 permanent
## 3 31     FDI          3 permanent
## 4 24     FDI          2 permanent
## 5 47     FDI          4 permanent

```

## 6 48	FDI	4 permanent
## 7 55	FDI	1 primary
## 8 65	FDI	2 primary

4. Data structuring: tidy_dental()

`tidy_dental()` identifies the structure of a dental dataset and converts it into a unified tidy format suitable for clinical index computation and statistical analysis. It automatically determines if the input is periodontal (wide or long), tooth-level, or sextant-level.

The function standardizes alphanumeric notation to FDI and keeps existing FDI or Universal codes unchanged. Ambiguous codes are not automatically interpreted to prevent errors; these should be resolved beforehand using specific package functions. To ensure data integrity, `tidy_dental()` triggers an error if invalid dental codes are detected or if the dataset contains mixed formats.

The transformation workflow includes:

- surface harmonization: standardizing surface names (e.g., converting 'v', 'V', 'b', or 'B' to 'B'; and 'p', 'l', or 'L' to 'L').
- reshaping: converting wide formats into a long tibble (one row per tooth-surface combination).
- data cleaning: removing duplicate rows and normalizing column names (e.g., replacing dots or hyphens with underscores).
- variable preservation: Maintaining all non-dental patient variables and summary-level indices (such as DMFT/DEF or PCR).

Non-dental variables such as the examiner are retained, and duplicated entries are removed.

If no specific dental structure is detected, the function normalizes column names, standardizes the patient identifier to `patient_id`, and returns the dataset with unique rows (`dplyr::distinct()`). This ensures that summary-level indices such as DMFT/DEF or ICON preserve their core data structure under a cleaned format without undergoing reshaping.

Example:

```
raw <- tibble(
  patient_id = rep(1:3, each = 4),

  # Alphanumeric inconsistent
  tooth = c("ur6", "UR6", "UL1", "l13",
            "UR6", "u11", "LL3", "lr7",
            "UR6", "UL1", "LL3", "LR7"),

  # VALID periodontal surfaces (numeric, no NA)
  m = c(1,0,1,0, 1,1,0,0, 0,1,1,0),
  d = c(0,1,0,1, 1,0,1,0, 0,0,1,1),
  v = c(1,0,0,1, 0,1,0,1, 1,0,0,1),
  p = c(0,1,1,0, 1,0,0,1, 1,1,0,0),

  examiner = sample(c("A", "B"), 12, replace = TRUE)
)

tidy <- tidy_dental(raw)
tidy
```

```
## # A tibble: 48 × 5
##   patient_id tooth examiner value tooth_side
##       <int> <chr> <chr>    <dbl> <chr>
## 1         1    1 16      A         1 M
## 2         2    1 16      A         0 D
## 3         3    1 16      A         1 B
## 4         4    1 16      A         0 L
## 5         5    1 16      A         0 M
## 6         6    1 16      A         1 D
## 7         7    1 16      A         0 B
## 8         8    1 16      A         1 L
## 9         9    1 21      B         1 M
## 10        10    1 21      B         0 D
## # i 38 more rows
```

In this example, `tidy_dental()` recognizes the input as a periodontal wide format (explicit tooth column alongside individual surface columns) and applies the full normalization workflow. Alphanumeric tooth codes are safely converted to FDI, surface names are harmonized to standard uppercase codes (M, D, B, L), and the dataset is reshaped into a clean long format with one row per tooth–surface combination.

5. Caries indices: DMFT and def

5.1. Normalization

The function `normalize_dmft_format()` standardizes caries data before computing the DMFT index. It accepts either a binary D/M/F scheme or a single status column and converts all inputs into a validated, mutually exclusive binary format. The function triggers an error if both a status column and D/M/F columns are present simultaneously. It ensures that only one coding system is used, that D/M/F values are coherent, and that the output is directly compatible with `compute_dmft()`. Status values (“D”, “M”, “F”, “S”, NA) are safely mapped to binary indicators, while binary inputs are checked for exclusivity and valid values. The function returns a tibble with `patient_id`, `tooth`, and the normalized D, M, and F columns.

Example:

```
teeth_perm <- c(as.character(11:17), as.character(21:27),
               as.character(31:37), as.character(41:47))

# Patient 1: status format
p1 <- tibble(
  patient_id = 1,
  tooth = teeth_perm,
  status = "S"
)
p1$status[p1$tooth == "11"] <- "D"
p1$status[p1$tooth == "16"] <- "F"
p1$status[p1$tooth == "26"] <- "M"

# Patient 2: binary format
p2 <- tibble(
  patient_id = 2,
  tooth = teeth_perm,
  D = 0, M = 0, F = 0
)
```

```

)
p2$D[p2$tooth == "21"] <- 1
p2$F[p2$tooth == "36"] <- 1
p2$M[p2$tooth == "47"] <- 1

# Patient 3: all sound
p3 <- tibble(
  patient_id = 3,
  tooth = teeth_perm,
  status = "S"
)

# Patient 4: two decayed, one filled
p4 <- tibble(
  patient_id = 4,
  tooth = teeth_perm,
  status = "S"
)
p4$status[p4$tooth == "12"] <- "D"
p4$status[p4$tooth == "37"] <- "D"
p4$status[p4$tooth == "46"] <- "F"

# Normalize all
df_all <- bind_rows(
  normalize_dmft_format(p1),
  normalize_dmft_format(p2),
  normalize_dmft_format(p3),
  normalize_dmft_format(p4)
)

# View the normalized result
head(df_all, 10)

```

```

## # A tibble: 10 × 5
##   patient_id tooth      D      M      F
##   <dbl> <chr> <dbl> <dbl> <dbl>
## 1         1 11      1      0      0
## 2         1 12      0      0      0
## 3         1 13      0      0      0
## 4         1 14      0      0      0
## 5         1 15      0      0      0
## 6         1 16      0      0      1
## 7         1 17      0      0      0
## 8         1 21      0      0      0
## 9         1 22      0      0      0
## 10        1 23      0      0      0

```

The function `normalize_def_format()` standardizes primary-teeth caries data before computing the def index. It accepts either a binary D/E/F scheme or a single status column and converts all inputs into a validated, mutually exclusive binary format. Status values ("D", "E", "F", "S", NA) are mapped to D/E/F indicators, while binary inputs are checked for exclusivity. The output contains `patient_id`, `tooth`, and the normalized D, E, F columns, ready for `compute_def()`.

Example:

```

teeth_primary <- c(as.character(51:55), as.character(61:65),
                  as.character(71:75), as.character(81:85))

# Patient 1: status format, three affected teeth
p1 <- tibble(
  patient_id = 1,
  tooth = teeth_primary,
  status = "S"
)
p1$status[p1$tooth == "51"] <- "D"
p1$status[p1$tooth == "62"] <- "E"
p1$status[p1$tooth == "85"] <- "F"

# Patient 2: binary format
p2 <- tibble(
  patient_id = 2,
  tooth = teeth_primary,
  D = 0, E = 0, F = 0
)
p2$D[p2$tooth == "54"] <- 1
p2$F[p2$tooth == "75"] <- 1

# Patient 3: all sound
p3 <- tibble(
  patient_id = 3,
  tooth = teeth_primary,
  status = "S"
)

# Patient 4: two decayed, one extracted
p4 <- tibble(
  patient_id = 4,
  tooth = teeth_primary,
  status = "S"
)
p4$status[p4$tooth == "52"] <- "D"
p4$status[p4$tooth == "64"] <- "D"
p4$status[p4$tooth == "83"] <- "E"

# Patient 5: one filled tooth
p5 <- tibble(
  patient_id = 5,
  tooth = teeth_primary,
  status = "S"
)
p5$status[p5$tooth == "71"] <- "F"

# Normalize all
df_def <- bind_rows(
  normalize_def_format(p1),
  normalize_def_format(p2),
  normalize_def_format(p3),
  normalize_def_format(p4),

```



```

    normalize_def_format(p5)
)

# View the normalized result
head(df_def, 10)

```

```

## # A tibble: 10 × 5
##   patient_id tooth      D      E      F
##   <dbl> <chr> <dbl> <dbl> <dbl>
## 1         1 1 51      1      0      0
## 2         2 1 52      0      0      0
## 3         3 1 53      0      0      0
## 4         4 1 54      0      0      0
## 5         5 1 55      0      0      0
## 6         6 1 61      0      0      0
## 7         7 1 62      0      1      0
## 8         8 1 63      0      0      0
## 9         9 1 64      0      0      0
## 10        10 1 65      0      0      0

```

5.2. Computing DMFT

The function `compute_dmft()` calculates the DMFT index for permanent dentition. It requires teeth to be coded in numeric FDI notation and uses the normalized binary D/M/F format. Patients are included only if all permanent teeth from second molar to second molar (11–17, 21–27, 31–37, 41–47) are present and valid. Any invalid tooth numbers, primary teeth, missing required teeth, or conflicting D/M/F values lead to the patient being excluded from the calculation. The function returns one DMFT value per valid patient.

Example:

```

# compute DMFT
compute_dmft(df_all)

```

```

## # A tibble: 4 × 2
##   patient_id dmft
##   <dbl> <int>
## 1         1     3
## 2         2     3
## 3         3     0
## 4         4     3

```

5.3. Computing def

`compute_def()` calculates the def index for primary dentition.

It includes strict validation:

- only primary FDI teeth
- complete dentition 51–55, 61–65, 71–75, 81–85

- invalid or incomplete patients are omitted with a single consolidated warning.

Example:

```
# Compute def
compute_def(df_def)

## # A tibble: 5 × 2
##   patient_id  def
##       <dbl> <int>
## 1         1     3
## 2         2     2
## 3         3     0
## 4         4     3
## 5         5     1
```

6. Periodontal indices

The `oralR` package includes a complete set of tools for computing the widely used periodontal indices: BOP, PCR, PI, and PSI. All indices follow strict clinical rules and rely on robust validation to ensure that the input data are structurally correct and clinically coherent.

Periodontal indices based on tooth or surface-level data (such as BOP, PI, and PCR) require tidy data; therefore, it is recommended to harmonise these datasets using `tidy_dental()` before validation and computation.

The package includes:

- `validate_index_input()`
- `compute_bop()` – Bleeding on Probing
- `compute_pcr()` – Plaque Control Record
- `compute_pi()` – Plaque Index
- `compute_psi()` – Periodontal Screening Index

6.1. Input validation: `validate_index_input()`

Before computing BOP, PCR or PI, the package applies a global validation step using `validate_index_input()`. This function ensures that:

- required columns exist (`patient_id`, `tooth`, `tooth_side`, and the clinical variable)
- tooth surfaces match the index definition

BOP: MB, B, DB, ML, L, DL
PCR/PI: M, D, B, L

- international synonyms are accepted

V / B (vestibular/buccal)
P / L (palatal/lingual)

- values are within the allowed range

BOP/PCR: 0-1
PI: integers 0-3

If any structural error is detected (missing columns, invalid surfaces, impossible values), the function stops with an informative message. If everything is correct, it returns TRUE invisibly.

`validate_index_input()` may be used manually, but `compute_bop()`, `compute_pi()`, and `compute_pcr()` call it automatically to guarantee consistent structural validation and tooth-side normalization.

Example:

```
# Validation will return TRUE invisibly if the dataset is correct
validate_index_input(tidy, index = "PCR")
```

6.2. Bleeding on Probing (BOP): `compute_bop()`

`compute_bop()` calculates the percentage of bleeding sites per patient. Each permanent tooth must contribute exactly six surfaces: MB, B, DB, ML, L, DL.

The function:

- validates the dataset with `validate_index_input()`
- removes rows with missing essential values
- checks that all teeth are permanent FDI teeth
- ensures no duplicated surfaces
- ensures each tooth has exactly six surfaces
- computes:
$$\text{total_points bleeding_points bop_percent} = \text{bleeding_points} / \text{total_points} * 100$$

Example:

```
df_bop_raw <- tibble::tribble(
  # Patient 1: valid (6 surfaces)
  ~patient_id, ~tooth, ~tooth_side, ~bop,
  1, "11", "MB", 1,
  1, "11", "B", 0,
  1, "11", "DB", 1,
  1, "11", "ML", 0,
  1, "11", "L", 1,
  1, "11", "DL", 0,

  # Patient 2: valid (no bleeding)
  2, "21", "MB", 0,
  2, "21", "B", 0,
  2, "21", "DB", 0,
  2, "21", "ML", 0,
  2, "21", "L", 0,
  2, "21", "DL", 0,

  # Patient 3: valid (high bleeding)
  3, "16", "MB", 1,
```

```

3, "16", "B", 1,
3, "16", "DB", 1,
3, "16", "ML", 1,
3, "16", "L", 1,
3, "16", "DL", 1,

# Patient 4: valid (mixed pattern)
4, "24", "MB", 0,
4, "24", "B", 1,
4, "24", "DB", 0,
4, "24", "ML", 1,
4, "24", "L", 0,
4, "24", "DL", 1,

# Patient 5: INVALID (missing one surface)
5, "36", "MB", 1,
5, "36", "B", 0,
5, "36", "DB", 1,
5, "36", "ML", 0,
5, "36", "L", 1
# DL surface missing: will be excluded
)

tidy_bop <- tidy_dental(df_bop_raw)
validate_index_input(tidy_bop, index = "BOP")
compute_bop(tidy_bop)

## Warning: Some patients were omitted due to invalid data:
## - Patient 5: Teeth with incorrect site counts (must be 6): 36

## # A tibble: 4 × 4
##   patient_id total_points bleeding_points bop_percent
##       <dbl>       <int>         <dbl>       <dbl>
## 1         1         6         3         50
## 2         2         6         0          0
## 3         3         6         6        100
## 4         4         6         3         50

```

This example illustrates the full workflow for BOP computation. The dataset is first standardized with `tidy_dental()`, then validated with `validate_index_input()`, and finally processed with `compute_bop()`. Patients 1–4 contain a complete set of six periodontal surfaces per permanent tooth and therefore produce valid BOP scores. Patient 5 is intentionally incomplete (missing the DL surface) and is automatically excluded during validation. This demonstrates how the package safeguards clinical calculations by detecting structural inconsistencies before computing the index.

6.3. Plaque Control Record (PCR): `compute_pcr()`

`compute_pcr()` computes the percentage of surfaces with plaque. Each tooth must have exactly four surfaces: M, D, B, L (with V/P synonyms allowed).

The function:

- validates the dataset
- removes rows with missing plaque values
- checks FDI notation
- ensures no duplicated surfaces
- ensures each tooth has exactly four surfaces
- computes:
 - total_points
 - plaque_points
 - pcr_percent

Example:

```
compute_pcr(tidy)
```

```
## Warning: Some patients were omitted due to invalid data:
## - Patient 1: Duplicated tooth-side entries detected.
```

```
## # A tibble: 2 × 4
##   patient_id total_points plaque_points pcr_percent
##       <int>         <int>         <dbl>         <dbl>
## 1         2             16             8             50
## 2         3             16             8             50
```

In this example, `tidy_dental()` correctly converts the dataset into periodontal long format, but the first patient contains duplicated tooth–surface combinations after Alphanumeric-to-FDI conversion. Specifically, the tooth 16 appears twice for patient 1, which produces repeated M, D, B and L surfaces once the dataset is pivoted. Because the Plaque Control Record requires exactly one value per surface, `compute_pcr()` automatically excludes this patient and reports the issue. The index is therefore computed only for patients with structurally valid data.

6.4. Plaque Index (PI): `compute_pi()`

`compute_pi()` computes the Silness & Loe Plaque Index, an ordinal score (0–3) averaged across four surfaces per tooth.

The function:

- validates structure and value ranges
- removes rows with missing values
- checks FDI notation
- ensures each tooth has exactly four surfaces
- ensures plaque values are integers 0–3
- computes:
 - total_sites: Total number of evaluated surfaces per patient
 - total_score: Cumulative sum of plaque scores per patient
 - pi_index: The final average Plaque Index (mean plaque score)

Example:

```
df_pi_raw <- tibble::tribble(
  # Patient 1: valid PI (mixed surface labels)
  ~patient_id, ~tooth, ~tooth_side, ~plaque,
  1, "ur1", "m", 0,
  1, "ur1", "d", 1,
  1, "ur1", "v", 2, # vestibular → B
  1, "ur1", "p", 3, # palatal → L

  # Patient 2: valid PI (FDI already correct)
  2, "21", "M", 1,
  2, "21", "D", 0,
  2, "21", "B", 2,
  2, "21", "L", 1,

  # Patient 3: valid PI
  3, "11", "M", 0,
  3, "11", "D", 1,
  3, "11", "B", 3,
  3, "11", "L", 2
)

# Step 1: tidy the dataset
tidy_pi <- tidy_dental(df_pi_raw)

# Step 2: compute PI
compute_pi(tidy_pi)

## # A tibble: 3 × 4
##   patient_id total_sites total_score pi_index
##       <dbl>       <int>       <dbl>   <dbl>
## 1         1         4         6     1.5
## 2         2         4         4         1
## 3         3         4         6     1.5
```

6.5. Periodontal Screening Index (PSI): `compute_psi()`

`compute_psi()` calculates the final Periodontal Screening Index (PSI) score per patient, which corresponds to the maximum score recorded across all evaluated sextants.

The function is highly flexible and dynamically handles two distinct input formats:

- Tooth-level data: Requires permanent FDI tooth numbers and `psi_code`. Each tooth is mapped to its corresponding standard sextant.
- Sextant-level data: Requires explicit sextant identifiers (1–6) and `psi_code`.

The operation mode is determined independently per patient, not globally for the entire dataset:

- If a patient record contains any non-NA value in the tooth column, tooth-level mode is triggered.
- If the tooth column is completely empty or NA, but values exist in the sextant column, sextant-level mode is triggered.

To ensure data integrity, the function adopts a zero-tolerance policy against data entry anomalies: the function omits the entire patient record and generates a detailed consolidated warning at the end of

execution.

A patient is omitted if any of the following are detected:

- Invalid tooth identifiers: Any code outside the permanent FDI range (11–18, 21–28, 31–38, 41–48), including typos or primary dentition codes (51–85).
- Invalid sextant identifiers: Values outside the standard 1–6 integer range.
- Out-of-bounds scores: Any `psi_code` value that is not an integer between 0 and 4.
- Missing clinical data: Patients with no non-NA `psi_code` values available.

Example:

The following example demonstrates how `compute_psi()` handles a mixed dataset containing valid, tooth-level, sextant-level, and invalid patient records:

```
# a mixed sample dataset
df_psi <- tibble::tribble(
  ~patient_id, ~tooth, ~sextant, ~psi_code,

  # Patient 1: Valid tooth-level mode (FDI permanent teeth)
  "P01",      "11",  NA,      2,
  "P01",      "12",  NA,      3,
  "P01",      "21",  NA,      1, # Max score = 3

  # Patient 2: Valid sextant-level mode
  "P02",      NA,    1,      1,
  "P02",      NA,    2,      0,
  "P02",      NA,    3,      4, # Max score = 4

  # Patient 3: Valid mixed input (tooth-level takes priority)
  "P03",      "16",  NA,      2,
  "P03",      NA,    1,      1, # Evaluated under tooth mode; NA rows are bypassed
  "P03",      NA,    2,      0, # Max score = 2

  # Patient 4: INVALID (triggers omission due to primary dentition code)
  "P04",      "54",  NA,      2,

  # Patient 5: INVALID (triggers omission due to out-of-bounds psi_code)
  "P05",      "11",  NA,      7
)
```

```
compute_psi(df_psi)
```

```
## Warning: Some patients were omitted due to invalid PSI data:
```

```
## - Patient P04: Invalid FDI tooth numbers: 54
```

```
## - Patient P05: PSI values must be between 0 and 4.
```

```
## # A tibble: 3 × 2
##   patient_id  psi
##   <chr>      <int>
## 1 P01        3
## 2 P02        4
```

7. Orthodontic index: ICON

The `oralR` package includes support for computing the Index of Complexity, Outcome and Need (ICON), following the official criteria of Daniels & Richmond (2000). ICON is an orthodontic index that evaluates:

- treatment need
- case complexity
- treatment outcome
- degree of improvement

Unlike periodontal indices, ICON does not require tooth-level or surface-level data. All computations operate on patient-level datasets, typically with one row per patient and per time point (e.g., pre and post).

The ICON workflow consists of two steps:

- `score_icon_components()`: validates raw clinical measurements and computes the five ICON component scores
- `compute_icon()`: applies official weightings, derives global ICON scores, and assigns all clinical classifications

7.1. Component scoring: `score_icon_components()`

`score_icon_components()` transforms raw clinical measurements into the five ICON component scores. This function applies strict validation to ensure that each record contains all required variables and that values fall within clinically acceptable ranges.

Required columns: Each row must include:

- `patient_id`
- `time` ("pre" or "post")
- `aesthetic_component`
- `upper_crowding_mm`, `upper_spacing_mm`
- `crossbite`
- `incisor_openbite_mm`, `incisor_overbite_category`
- `buccal_ap_left`, `buccal_ap_right`
- `impacted_teeth`

Component rules: The function validates and computes:

- Aesthetic Component (1–10)
- Upper Arch Crowding/Spacing Score (0–5)
at least one of crowding or spacing must be provided values must be ≥ 0 and $\leq$ `max_discrepancy_allowed` (default 20 mm) if both exist, the larger is used if `impacted_teeth = TRUE`, the score is automatically set to 5
- Crossbite Score (0–1) logical; numeric/text values are converted when possible
- Vertical Score (0–4)

based on open bite (mm) and/or overbite category (0–3)
vertical score = max(openbite_score, overbite_score)

- Buccal AP Score (0–4)

sum of left and right sides (each 0–2).

Error handling: Two modes are available:

- `.on_error = "stop"` — stops immediately on the first invalid row.
- `.on_error = "collect"` (default) — processes valid rows and returns a list with:

scores – tibble of valid ICON component scores.
errors – tibble of excluded rows and reasons.

Returned values: For each valid row, the function returns:

- patient_id, time
- aesthetic_component
- upper_arch_crowding_score
- crossbite_score
- vertical_score
- buccal_ap_score

`compute_icon()` always runs `score_icon_components()` internally; users do not need to call it manually.

Example:

```
df_icon <- tibble::tribble(  
  ~patient_id, ~time,  
  ~aesthetic_component,  
  ~upper_crowding_mm, ~upper_spacing_mm,  
  ~crossbite,  
  ~incisor_openbite_mm, ~incisor_overbite_category,  
  ~buccal_ap_left, ~buccal_ap_right,  
  ~impacted_teeth,  
  
  # Patient 1: crowding, mild open bite  
  1, "pre",  
  6,  
  4, NA,  
  FALSE,  
  1.5, NA,  
  1, 1,  
  FALSE,  
  
  # Patient 1: post-treatment  
  1, "post",  
  4,  
  1, NA,  
  FALSE,  
  0, 1,  
  0, 1,
```

```

FALSE,

# Patient 2: spacing, deep overbite
2, "pre",
8,
NA, 6,
TRUE,
NA, 3,
2, 2,
FALSE,

# Patient 3: impacted teeth
3, "pre",
7,
2, 1,
FALSE,
0, 0,
1, 0,
TRUE
)

result_icon <- score_icon_components(df_icon)
result_icon$scores

## # A tibble: 4 × 7
##   patient_id time aesthetic_component upper_arch_crowding_score crossbite_score
##       <dbl> <chr>                <dbl>                <int>          <int>
## 1         1 pre                    6                    1            0
## 2         1 post                    4                    0            0
## 3         2 pre                    8                    2            1
## 4         3 pre                    7                    5            0
## # i 2 more variables: vertical_score <int>, buccal_ap_score <int>

```

7.2. Global ICON scoring: compute_icon()

`compute_icon()` aggregates the component scores produced by `score_icon_components()`. The function automatically calls `score_icon_components()` internally to ensure that all component scores are valid. It computes:

- global ICON scores (pre and post)
- treatment need
- complexity grade
- outcome acceptability
- improvement grade

The global ICON score is computed as:

- Aesthetic Component × 7
- Crossbite Score × 5
- Upper Arch Crowding/Spacing Score × 5

- Vertical Score $\times 4$
- Buccal AP Score $\times 3$

The function returns:

- icon_pre
- icon_post
- icon_improvement = icon_pre - 4 * icon_post

Clinical classifications:

1. Treatment need:

- TRUE if icon_pre ≥ 43
- FALSE if icon_pre < 43

2. Complexity grade:

(based on icon_pre)

Score (icon_pre)	Complexity Grade
≤ 29	Easy
30 – 50	Mild
51 – 63	Moderate
64 – 77	Difficult
> 77	Very difficult

3. Outcome acceptability:

Acceptable if icon_post < 31

4. Improvement grade

Based on the improvement index: $I = \text{icon_pre} - 4 \times \text{icon_post}$

Categories:

I	Improvement Grade
$I > -1$	Greatly improved
$-25 \leq I \leq -1$	Substantially improved
$-53 \leq I \leq -26$	Moderately improved
$-85 \leq I \leq -54$	Minimally improved
$I < -85$	Not improved or worse

Example:

```
result_final <- compute_icon(df_icon)
result_final$scores
```

```
## # A tibble: 3 × 8
```

```
##   patient_id icon_pre icon_post icon_improvement treatment_need complexity_grade
```

```
##           <dbl>    <dbl>    <dbl>           <dbl> <lgl>           <fct>
## 1           1       61       35           -79 TRUE       Moderate
## 2           2       95       NA            NA TRUE       Very difficult
## 3           3       77       NA            NA TRUE       Difficult
## # i 2 more variables: outcome_acceptable <lgl>, improvement_grade <fct>
```

8. Statistical tools

The `oralR` package includes two general-purpose statistical functions designed to complement the clinical indices: `analyze_descriptives()` and `analyze_prevalence()`. These functions are independent of dental notation and can be applied to any numeric or binary variable, making them useful for clinical reporting, epidemiological summaries and exploratory data analysis.

8.1. Descriptive statistics: `analyze_descriptives()`

`analyze_descriptives()` generates a structured statistical summary of a numeric vector. Unlike the clinical index functions, it does not validate dental data; it operates exclusively on already computed numeric values (e.g., DMFT, def, PI, PSI, ICON scores, age, treatment duration). This function automatically computes basic statistics, dispersion measures, a normality test and outlier detection using the IQR rule.

The function computes:

- basic statistics: n, mean, median, sd
- dispersion: min, max, p25, p75
- Shapiro–Wilk normality test (when applicable)
- outlier detection using Tukey's rule ($1.5 \times \text{IQR}$)

Input: A single argument: `x`: numeric vector (or convertible to numeric) NA values are removed automatically.

Details:

1. Central tendency and dispersion. The function returns: n, mean, median, sd, min, max, p25, p75.
2. Normality test. Shapiro–Wilk test is applied when:
 - $3 \leq n \leq 5000$ variance > 0 If variance is zero, normality_p = NA.
3. Outlier detection
 - Outliers are values $< Q1 - 1.5 \times \text{IQR}$ or $> Q3 + 1.5 \times \text{IQR}$ Returned as a comma-separated string If none detected or $n < 4$: outliers = NA
4. Robust input handling
 - character vectors convertible to numeric are accepted empty or all-NA vectors return $n = 0$ and all other fields as NA

Example:

```
# DMFT: descriptive statistics with previous example
dmft_results <- compute_dmft(df_all)
analyze_descriptives(dmft_results$dmft)
```

```
## # A tibble: 1 × 10
```

```
##           n mean median    sd  min  max  p25  p75 normality_p outliers
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
## 1     4  2.25     3   1.5    0    3  2.25    3    0.00124 0
```

```
# def: descriptive statistics with previous example
def_results <- compute_def(df_def)
analyze_descriptives(def_results$def)
```

```
## # A tibble: 1 × 10
##           n mean median    sd  min  max  p25  p75 normality_p outliers
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
## 1     5  1.8     2  1.30    0    3    1    3    0.421 <NA>
```

8.2. Prevalence: analyze_prevalence()

`analyze_prevalence()` computes the prevalence of a binary clinical condition. It is designed to complement clinical indices such as BOP, PCR, PI, PSI, DMFT components, or any variable representing presence/absence. This function calculates counts, prevalence and a Wilson confidence interval, and handles non-numeric or mixed input robustly.

The function returns:

- `n`: number of valid observations
- `n_positive`: number of TRUE/positive cases
- `prevalence`: proportion of positives
- `ci_low`, `ci_high`: 95% Wilson confidence interval

Input: A single argument: `x`: binary vector (logical, numeric, or convertible)

Accepted formats:

- logical (TRUE/FALSE)
- numeric (0/1, or any non-zero interpreted as TRUE)
- character values convertible to logical ("TRUE", "FALSE", "T", "F")
- NA values are ignored.

Details:

- Binary interpretation
 - 0: FALSE any non-zero numeric: TRUE non-convertible values: NA
- Handling NA
 - removed before computation if all values are NA, returns `n = 0` and all other fields as NA
- Confidence interval
 - Wilson method (robust for extreme prevalences) only computed when `n > 0`

Example:

```
# DMFT binary indicator: patient has at least one affected permanent tooth
dmft_binary <- dmft_results$dmft > 0
analyze_prevalence(dmft_binary)
```

```
## # A tibble: 1 × 5
##       n n_positive prevalence ci_low ci_high
##   <int>      <int>      <dbl> <dbl>   <dbl>
## 1     4          3        0.75  0.301   0.954

# def binary indicator: patient has at least one affected primary tooth
def_binary <- def_results$def > 0
analyze_prevalence(def_binary)

## # A tibble: 1 × 5
##       n n_positive prevalence ci_low ci_high
##   <int>      <int>      <dbl> <dbl>   <dbl>
## 1     5          4        0.8    0.376   0.964
```

9. Complete example workflow (9 synthetic patients).

This section illustrates a full analysis workflow using `oralR`, from raw heterogeneous data to validated clinical indices, descriptive statistics and graphical summaries. To demonstrate the different modules of the package, we generate four synthetic datasets for the same 9 patients:

- Tooth-level dataset (for DMFT)
- Surface-level periodontal dataset (for BOP, PCR, PI)
- Tooth- or sextant-level dataset (for PSI)
- Patient-level dataset (for ICON)

These datasets are intentionally simple but structurally realistic, allowing all major functions of the package to be demonstrated in a single workflow.

```
# Dental coding workflow (synthetic mixed-notation dataset)
set.seed(123)

# 9 synthetic patients
patients <- paste0("P", sprintf("%02d", 1:9))

# Permanent teeth required for DMFT
teeth_perm <- c(as.character(11:17), as.character(21:27),
               as.character(31:37), as.character(41:47))

# Patients 1-3: FDI notation
df_fdi <- bind_rows(lapply(patients[1:3], function(id) {
  tibble(
    patient_id = id,
    notation_original = "FDI",
    tooth_original = teeth_perm
  )
}))

# Patients 4-6: Universal notation
```

```
df_universal <- bind_rows(lapply(patients[4:6], function(id) {
  tibble(
    patient_id = id,
    notation_original = "Universal",
    tooth_original = convert_fdi_to_universal(teeth_perm)
  )
}))
```

Patients 7-9: Alphanumeric notation

```
df_alpha <- bind_rows(lapply(patients[7:9], function(id) {
  tibble(
    patient_id = id,
    notation_original = "Alphanumeric",
    tooth_original = convert_fdi_to_alphanumeric(teeth_perm)
  )
}))
```

Combine all patients

```
df_codes <- bind_rows(df_fdi, df_universal, df_alpha)
```

Displaying the first tooth for each patient:

```
df_codes %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 3
```

```
##   patient_id notation_original tooth_original
##   <chr>      <chr>             <chr>
## 1 P01      FDI              11
## 2 P02      FDI              11
## 3 P03      FDI              11
## 4 P04      Universal         8
## 5 P05      Universal         8
## 6 P06      Universal         8
## 7 P07      Alphanumeric      UR1
## 8 P08      Alphanumeric      UR1
## 9 P09      Alphanumeric      UR1
```

9.1. Detecting notation

```
df_detect <- df_codes %>%
  mutate(notation_detected = detect_notation(tooth_original))

# Displaying the first tooth for each patient:
df_detect %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()

## # A tibble: 9 × 4
##   patient_id notation_original tooth_original notation_detected
##   <chr>      <chr>            <chr>          <list>
## 1 P01      FDI              11            <chr [2]>
## 2 P02      FDI              11            <chr [2]>
## 3 P03      FDI              11            <chr [2]>
## 4 P04      Universal          8            <chr [1]>
## 5 P05      Universal          8            <chr [1]>
## 6 P06      Universal          8            <chr [1]>
## 7 P07      Alphanumeric        UR1           <chr [1]>
## 8 P08      Alphanumeric        UR1           <chr [1]>
## 9 P09      Alphanumeric        UR1           <chr [1]>
```

9.2. Converting all tooth codes to FDI notation

```
df_converted <- df_detect %>%
  mutate(
    tooth_fdi = case_when(
      notation_original == "FDI" ~ tooth_original,
      notation_original == "Universal" ~ convert_universal_to_fdi(tooth_original),
      notation_original == "Alphanumeric" ~ convert_alphanumeric_to_fdi(tooth_original),
      TRUE ~ NA_character_
    )
  )

# Displaying the first tooth for each patient:
df_converted %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()

## # A tibble: 9 × 5
##   patient_id notation_original tooth_original notation_detected tooth_fdi
##   <chr>      <chr>            <chr>          <list>      <chr>
## 1 P01      FDI              11            <chr [2]>    11
## 2 P02      FDI              11            <chr [2]>    11
## 3 P03      FDI              11            <chr [2]>    11
## 4 P04      Universal          8            <chr [1]>    11
## 5 P05      Universal          8            <chr [1]>    11
## 6 P06      Universal          8            <chr [1]>    11
```



```
## 7 P07      Alphanumeric      UR1      <chr [1]>      11
## 8 P08      Alphanumeric      UR1      <chr [1]>      11
## 9 P09      Alphanumeric      UR1      <chr [1]>      11
```

```
df_clean <- df_converted %>%
  select(patient_id, tooth_fdi)
```

Displaying the first tooth for each patient:

```
df_clean %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 2
##   patient_id tooth_fdi
##   <chr>      <chr>
## 1 P01      11
## 2 P02      11
## 3 P03      11
## 4 P04      11
## 5 P05      11
## 6 P06      11
## 7 P07      11
## 8 P08      11
## 9 P09      11
```

9.3. Computing DMFT per patient

Assigning tooth status (D/M/F/S) for DMFT analysis:

```
set.seed(123)
df_status <- df_clean %>%
  mutate(
    status = sample(c("D", "M", "F", "S", NA), size = n(), replace = TRUE)
  )
```

Display the initial structure before normalization

```
df_status %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 3
##   patient_id tooth_fdi status
##   <chr>      <chr>    <chr>
## 1 P01      11      F
## 2 P02      11      M
## 3 P03      11      M
## 4 P04      11      S
```

```
## 5 P05      11      D
## 6 P06      11      M
## 7 P07      11     <NA>
## 8 P08      11      D
## 9 P09      11      D
```

Normalizing DMFT format:

```
df_norm <- df_status %>%
  rename(tooth = tooth_fdi) %>%
  normalize_dmft_format()

# Display the structure after normalization
df_norm %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 5
##   patient_id tooth      D      M      F
##   <chr>      <chr> <dbl> <dbl> <dbl>
## 1 P01      11      0      0      1
## 2 P02      11      0      1      0
## 3 P03      11      0      1      0
## 4 P04      11      0      0      0
## 5 P05      11      1      0      0
## 6 P06      11      0      1      0
## 7 P07      11      0      0      0
## 8 P08      11      1      0      0
## 9 P09      11      1      0      0
```

Computing DMFT per patient:

```
df_dmft <- df_norm %>%
  compute_dmft()
```

```
df_dmft
```

```
## # A tibble: 9 × 2
##   patient_id dmft
##   <chr>      <int>
## 1 P01      20
## 2 P02      17
## 3 P03      18
## 4 P04      16
## 5 P05      13
## 6 P06      18
## 7 P07      18
## 8 P08      15
## 9 P09      19
```

9.4. Computing def per patient

Creating a synthetic dataset with FDI primary teeth:

```
# Primary teeth required for def
teeth_primary <- c(as.character(51:55), as.character(61:65),
                  as.character(71:75), as.character(81:85))

# Synthetic dataset for 9 patients with primary dentition
df_primary <- bind_rows(lapply(patients, function(id) {
  tibble(
    patient_id = id,
    tooth_fdi = teeth_primary
  )
}))
```

Assigning tooth status (d/e/f/s) for DEF analysis:

```
set.seed(123)
df_status_def <- df_primary %>%
  mutate(
    status = sample(c("D", "E", "F", "S", NA), size = n(), replace = TRUE)
  )

# Display the initial structure before normalization
df_status_def %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 3
##   patient_id tooth_fdi status
##   <chr>      <chr>    <chr>
## 1 P01       51      F
## 2 P02       51      E
## 3 P03       51    <NA>
## 4 P04       51    <NA>
## 5 P05       51      S
## 6 P06       51    <NA>
## 7 P07       51      D
## 8 P08       51      E
## 9 P09       51      F
```

Normalizing def format:

```
df_norm_def <- df_status_def %>%
  rename(tooth = tooth_fdi) %>%
  normalize_def_format()
```

```
# Display the structure after normalization
```

```
df_norm_def %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()

## # A tibble: 9 × 5
##   patient_id tooth      D      E      F
##   <chr>      <chr> <dbl> <dbl> <dbl>
## 1 P01        51      0      0      1
## 2 P02        51      0      1      0
## 3 P03        51      0      0      0
## 4 P04        51      0      0      0
## 5 P05        51      0      0      0
## 6 P06        51      0      0      0
## 7 P07        51      1      0      0
## 8 P08        51      0      1      0
## 9 P09        51      0      0      1
```

Computing def per patient:

```
df_def <- df_norm_def %>%
  compute_def()

df_def

## # A tibble: 9 × 2
##   patient_id  def
##   <chr>      <int>
## 1 P01        15
## 2 P02        13
## 3 P03        12
## 4 P04        13
## 5 P05        11
## 6 P06        10
## 7 P07        10
## 8 P08        12
## 9 P09        16
```

9.5. Computing BOP

This section illustrates how to compute the Bleeding on Probing (BOP) index using a synthetic periodontal dataset.

Creating a synthetic dataset:

First, we generate a periodontal dataset in wide format, where each row corresponds to a single tooth and each of the six periodontal surfaces is represented as a separate column. Each surface is assigned a binary BOP value (0 = no bleeding, 1 = bleeding).

```
set.seed(123)
```

```
df_perio <- df_clean %>%
  rename(tooth = tooth_fdi) %>%
  mutate(
    MB = sample(c(0, 1), size = n(), replace = TRUE),
    B  = sample(c(0, 1), size = n(), replace = TRUE),
    DB = sample(c(0, 1), size = n(), replace = TRUE),
    ML = sample(c(0, 1), size = n(), replace = TRUE),
    L  = sample(c(0, 1), size = n(), replace = TRUE),
    DL = sample(c(0, 1), size = n(), replace = TRUE)
  )
```

Displaying the first tooth for each patient:

```
df_perio %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 8
##   patient_id tooth    MB      B    DB    ML      L    DL
##   <chr>      <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 P01        11      0      1      0      1      1      0
## 2 P02        11      0      1      1      0      0      1
## 3 P03        11      1      0      0      0      1      1
## 4 P04        11      1      1      1      1      0      1
## 5 P05        11      1      1      1      1      0      1
## 6 P06        11      1      0      1      1      1      1
## 7 P07        11      1      0      0      0      1      0
## 8 P08        11      0      1      1      0      0      0
## 9 P09        11      1      0      0      1      0      1
```

Converting to tidy format:

The `tidy_dental()` function converts this wide periodontal table into a tidy long format, producing one row per tooth–surface combination and standardizing the structure for index computation.

```
df_tidy <- df_perio %>%
  tidy_dental()
```

Displaying the first six rows of the tidy dataset for patient P01:

```
df_tidy %>%
  filter(patient_id == "P01") %>%
  slice_head(n = 6) %>%
  ungroup()
```

```
## # A tibble: 6 × 4
##   patient_id tooth value tooth_side
##   <chr>      <chr> <dbl> <chr>
## 1 P01        11      0 MB
## 2 P01        11      1 B
## 3 P01        11      0 DB
## 4 P01        11      1 ML
## 5 P01        11      1 L
## 6 P01        11      0 DL
```

Validating input:

Before computing the index, we verify that the dataset satisfies all structural requirements:

- valid FDI tooth codes
- exactly six surfaces per tooth
- no duplicated entries
- BOP values restricted to 0, 1, or NA

You may call `validate_index_input()` manually, but this is not required: `compute_bop()` runs it automatically and internally, ensuring that global structural checks and tooth-side normalization (e.g., V into B, P into L) are always applied before the index-specific validation.

If the dataset is valid, the function returns TRUE invisibly.

```
validate_index_input(df_tidy, index = "bop")
```

Computing BOP:

Finally, we compute the BOP index for each patient. The function returns:

- `total_points`: total number of periodontal surfaces
- `bleeding_points`: number of surfaces with BOP = 1
- `bop_percent`: percentage of bleeding surfaces

```
df_bop <- compute_bop(df_tidy)
```

```
df_bop
```

```
## # A tibble: 9 × 4
##   patient_id total_points bleeding_points bop_percent
##   <chr>      <int>      <dbl>      <dbl>
## 1 P01        168        80        47.6
## 2 P02        168        80        47.6
## 3 P03        168        82        48.8
## 4 P04        168        74        44.0
## 5 P05        168        85        50.6
## 6 P06        168        84        50
## 7 P07        168        81        48.2
## 8 P08        168        87        51.8
## 9 P09        168        89        53.0
```

9.6. Computing PCR

This section illustrates how to compute the Plaque Control Record (PCR) using a synthetic dataset.

Creating a synthetic dataset:

First, we generate a dataset in wide format, where each row corresponds to a single tooth and the four plaque surfaces (M, D, B, L) are represented as separate columns.

```
set.seed(123)

df_pcr <- df_clean %>%
  rename(tooth = tooth_fdi) %>%      # Standard tooth column name
  mutate(
    M = sample(c(0, 1), size = n(), replace = TRUE),
    D = sample(c(0, 1), size = n(), replace = TRUE),
    B = sample(c(0, 1), size = n(), replace = TRUE),
    L = sample(c(0, 1), size = n(), replace = TRUE)
  )

# Displaying the first tooth for each patient:
df_pcr %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()

## # A tibble: 9 × 6
##   patient_id tooth      M      D      B      L
##   <chr>      <chr> <dbl> <dbl> <dbl> <dbl>
## 1 P01        11      0      1      0      1
## 2 P02        11      0      1      1      0
## 3 P03        11      1      0      0      0
## 4 P04        11      1      1      1      1
## 5 P05        11      1      1      1      1
## 6 P06        11      1      0      1      1
## 7 P07        11      1      0      0      0
## 8 P08        11      0      1      1      0
## 9 P09        11      1      0      0      1
```

Converting to tidy format:

The `tidy_dental()` function converts this wide periodontal table into a tidy long format, producing one row per tooth–surface combination and standardizing the structure for index computation.

```
df_tidy_pcr <- df_pcr %>%
  tidy_dental()

# Displaying the first four rows of the tidy dataset for patient P01:
df_tidy_pcr %>%
  filter(patient_id == "P01") %>%
  slice_head(n = 4) %>%
  ungroup()
```

```
## # A tibble: 4 × 4
##   patient_id tooth value tooth_side
##   <chr>      <chr> <dbl> <chr>
## 1 P01        11      0 M
## 2 P01        11      1 D
## 3 P01        11      0 B
## 4 P01        11      1 L
```

Validating input:

Before computing the index, we verify that the dataset satisfies all structural requirements:

- valid FDI tooth codes
- exactly four surfaces per tooth
- no duplicated entries
- PCR values restricted to 0, 1, or NA

You may call `validate_index_input()` manually, but this is not required: `compute_pcr()` runs it automatically and internally, ensuring that global structural checks and tooth-side normalization (e.g., V into B, P into L) are always applied before the index-specific validation.

If the dataset is valid, the function returns TRUE invisibly.

```
validate_index_input(df_tidy_pcr, index = "pcr")
```

Computing PCR:

Finally, we compute the PCR index for each patient. The function returns:

- `total_points`: total number of periodontal surfaces
- `plaque_points`: number of surfaces with PCR = 1
- `pcr_percent`: percentage of plaque-positive surfaces

```
df_pcr_result <- compute_pcr(df_tidy_pcr)
```

```
df_pcr_result
```

```
## # A tibble: 9 × 4
##   patient_id total_points plaque_points pcr_percent
##   <chr>      <int>      <dbl>      <dbl>
## 1 P01        112        50        44.6
## 2 P02        112        52        46.4
## 3 P03        112        55        49.1
## 4 P04        112        49        43.8
## 5 P05        112        62        55.4
## 6 P06        112        50        44.6
## 7 P07        112        60        53.6
## 8 P08        112        59        52.7
## 9 P09        112        61        54.5
```


9.7. Computing PI

This section demonstrates how to compute the Silness & Løe Plaque Index (PI) using a synthetic dataset.

Creating a synthetic dataset:

First, we generate a dataset in wide format, where each row corresponds to a single tooth and the four plaque-scoring surfaces (M, D, B/V, L/P) are represented as separate columns with ordinal values from 0 to 3.

```
set.seed(123)

df_pi <- df_clean %>%
  rename(tooth = tooth_fdi) %>%      # Standard tooth column name
  mutate(
    M = sample(0:3, size = n(), replace = TRUE),
    D = sample(0:3, size = n(), replace = TRUE),
    B = sample(0:3, size = n(), replace = TRUE),
    L = sample(0:3, size = n(), replace = TRUE)
  )

# Displaying the first tooth for each patient:
df_pi %>%
  group_by(patient_id) %>%
  slice_head(n = 1) %>%
  ungroup()
```

```
## # A tibble: 9 × 6
##   patient_id tooth      M      D      B      L
##   <chr>      <chr> <int> <int> <int> <int>
## 1 P01        11      2      1      2      1
## 2 P02        11      2      3      1      0
## 3 P03        11      1      0      2      2
## 4 P04        11      1      3      1      1
## 5 P05        11      3      1      3      3
## 6 P06        11      3      2      3      3
## 7 P07        11      1      2      2      0
## 8 P08        11      2      1      1      2
## 9 P09        11      3      0      0      3
```

Converting to tidy format:

`tidy_dental()` converts this wide PI table into a tidy long format, producing one row per tooth–surface combination and standardizing the structure for PI computation.

```
df_tidy_pi <- df_pi %>%
  tidy_dental()

# Displaying the first four rows of the tidy dataset for patient P01:
df_tidy_pi %>%
  filter(patient_id == "P01") %>%
```

```
slice_head(n = 4) %>%
ungroup()
```

```
## # A tibble: 4 × 4
##   patient_id tooth value tooth_side
##   <chr>      <chr> <int> <chr>
## 1 P01        11      2 M
## 2 P01        11      1 D
## 3 P01        11      2 B
## 4 P01        11      1 L
```

Validating input:

Before computing the index, we verify that the dataset satisfies all structural requirements:

- valid FDI tooth codes
- exactly four surfaces per tooth
- no duplicated entries
- PI values restricted plaque values restricted to 0–3 or NA.

You may call `validate_index_input()` manually, but this is not required: `compute_pi()` runs it automatically and internally, ensuring that global structural checks and tooth-side normalization (e.g., V into B, P into L) are always applied before the index-specific validation.

If the dataset is valid, the function returns TRUE invisibly.

```
validate_index_input(df_tidy_pi, index = "pi")
```

Computing PI:

The output includes:

- `total_points`: total number of examined surfaces
- `pi_mean`: mean PI score per patient

```
df_pi_result <- compute_pi(df_tidy_pi)
```

```
df_pi_result
```

```
## # A tibble: 9 × 4
##   patient_id total_sites total_score pi_index
##   <chr>      <int>      <int>    <dbl>
## 1 P01        112        156     1.39
## 2 P02        112        166     1.48
## 3 P03        112        155     1.38
## 4 P04        112        151     1.35
## 5 P05        112        176     1.57
## 6 P06        112        184     1.64
## 7 P07        112        178     1.59
## 8 P08        112        185     1.65
## 9 P09        112        161     1.44
```

9.8. Computing PSI

The Periodontal Screening Index (PSI), also known as the Community Periodontal Index (CPI), summarizes the periodontal condition of each patient using the maximum score across the six sextants. Unlike the other periodontal indices in this vignette, PSI does not rely on `tidy_dental()` or `validate_index_input()`, because it accepts two different input formats and determines the processing mode per patient.

Creating a synthetic dataset:

First, we generate a dataset containing both tooth-level and sextant-level PSI observations, and also several patients with invalid entries to illustrate how `compute_psi()` handles errors and determines the processing mode for each patient.

```
df_psi <- tibble::tribble(  
  # Patients 1-4: tooth-level data (valid)  
  ~patient_id, ~tooth, ~sextant, ~psi_code,  
  1, "11", NA, 2,  
  1, "12", NA, 3,  
  1, "21", NA, 1,  
  1, "22", NA, 0,  
  
  2, "31", NA, 1,  
  2, "32", NA, 2,  
  2, "41", NA, 0,  
  2, "42", NA, 1,  
  
  3, "13", NA, 0,  
  3, "14", NA, 1,  
  3, "23", NA, 2,  
  3, "24", NA, 2,  
  
  4, "33", NA, 3,  
  4, "34", NA, 2,  
  4, "43", NA, 1,  
  4, "44", NA, 0,  
  
  # Patients 5-6: sextant-level data (valid)  
  5, NA, 1, 1,  
  5, NA, 2, 0,  
  5, NA, 3, 2,  
  5, NA, 4, 1,  
  
  6, NA, 1, 0,  
  6, NA, 2, 0,  
  6, NA, 3, 1,  
  6, NA, 4, 0,  
  
  # Patients 7-9: invalid data (to trigger warnings)  
  7, "11", NA, 7,      # invalid psi_code  
  8, "A3", NA, 2,      # invalid tooth format  
  9, NA, 9, 1          # invalid sextant  
)
```

```
# patients 1 (tooth-level data) and 5 (sextant-level data) as representative examples
df_psi %>%
  filter(patient_id %in% c(1, 5))

## # A tibble: 8 × 4
##   patient_id tooth sextant psi_code
##       <dbl> <chr>   <dbl>   <dbl>
## 1         1 11      NA       2
## 2         1 12      NA       3
## 3         1 21      NA       1
## 4         1 22      NA       0
## 5         5 <NA>    1       1
## 6         5 <NA>    2       0
## 7         5 <NA>    3       2
## 8         5 <NA>    4       1
```

Computing PSI:

The output includes:

- patient_id
- psi: PSI score per patient

```
result_psi <- compute_psi(df_psi)
```

```
## Warning: Some patients were omitted due to invalid PSI data:
## - Patient 7: PSI values must be between 0 and 4.
## - Patient 8: Invalid FDI tooth numbers: A3
## - Patient 9: Invalid sextant identifiers (must be 1-6): 9
```

```
result_psi
```

```
## # A tibble: 6 × 2
##   patient_id  psi
##       <dbl> <int>
## 1         1     3
## 2         2     2
## 3         3     2
## 4         4     3
## 5         5     2
## 6         6     1
```

Note: ICON orthodontic index follows a different workflow and is described separately in Section 7.

9.9. Statistical tools

ora1R provides two general-purpose statistical utilities: `analyze_descriptives()` and `analyze_prevalence()`. These functions can be applied either to raw clinical data (e.g., D = 0/1 per tooth) or to patient-level index results (e.g., DMFT, BOP, PCR, PI). The user is expected to choose the

appropriate level of analysis depending on the study design and the clinical question. These tools are not applicable to ordinal screening indices such as PSI or to composite orthodontic scores such as ICON.

Index / Data type	Output type	analyze_descriptives	analyze_prevalence
DMFT (patient)	count	yes	no
def (patient)	count	yes	no
D / M / F (tooth)	binary	no	yes
BOP (surface)	binary	no	yes
BOP (patient)	percentage	yes	no*
PCR (surface)	binary	no	yes
PCR (patient)	percentage	yes	no*
PI (patient)	mean score	yes	no
PSI	ordinal max	no	no
ICON	composite	no	no

**Note: Patient-level percentages are continuous metrics and must be summarized using `analyze_descriptives()` (mean, SD, medians). Prevalence calculations within `oralR` are strictly designed for logical or binary (0/1) tooth- or surface-level indicators representing presence/absence.*

```
analyze_descriptives(df_dmft$dmft)
```

```
## # A tibble: 1 × 10
##       n mean median   sd  min  max  p25  p75 normality_p outliers
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>      <dbl> <chr>
## 1     9  17.1    18  2.15   13   20   16   18      0.653 <NA>
```

```
analyze_prevalence(df_norm$D == 1)
```

```
## # A tibble: 1 × 5
##       n n_positive prevalence ci_low ci_high
##   <int>      <int>      <dbl> <dbl>   <dbl>
## 1   252         58      0.230  0.182  0.286
```

```
analyze_prevalence(df_norm_def$D == 1)
```

```
## # A tibble: 1 × 5
##       n n_positive prevalence ci_low ci_high
##   <int>      <int>      <dbl> <dbl>   <dbl>
## 1   180         41      0.228  0.173  0.294
```

```
analyze_descriptives(df_pi_result$pi_mean)
```

```
## # A tibble: 1 × 10
##       n mean median    sd min  max  p25  p75 normality_p outliers
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>      <dbl> <chr>
## 1     0    NA    NA    NA    NA    NA    NA    NA        NA <NA>
```

```
analyze_prevalence(df_bop$bop_percent > 50)
```

```
## # A tibble: 1 × 5
##       n n_positive prevalence ci_low ci_high
##   <int>      <int>      <dbl> <dbl>   <dbl>
## 1     9          3      0.333 0.121  0.646
```

```
analyze_prevalence(df_pcr_result$pcr_percent > 50)
```

```
## # A tibble: 1 × 5
##       n n_positive prevalence ci_low ci_high
##   <int>      <int>      <dbl> <dbl>   <dbl>
## 1     9          4      0.444 0.189  0.733
```

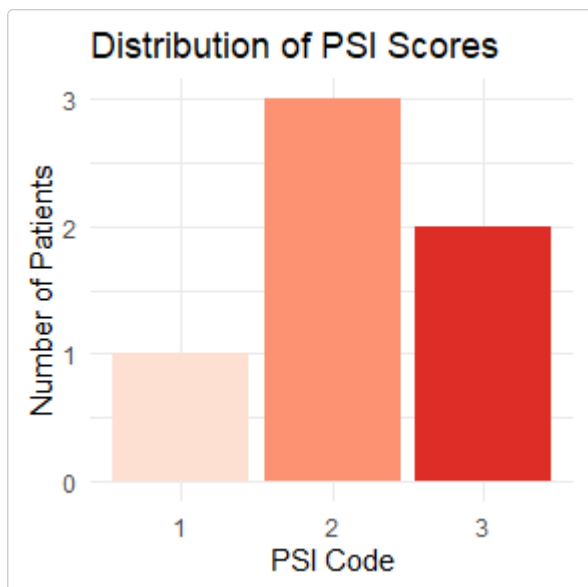
10. Integration with statistical workflows

The output generated by `tidy_dental()` and the various `compute_*` functions is designed to be compatible with standard R ecosystems, such as `tidyverse` and `stats`. Since the package returns standardized tibbles, users can directly pipe the results into visualization tools and statistical tests without additional manipulation.

- visualization: The long-format structure is ideal for `ggplot2`.
- statistical testing: The numerical outputs can be used in non-parametric tests like the Wilcoxon signed-rank test (for comparing pre- and post-treatment indices) or Spearman's rank correlation (to explore relationships between different clinical markers).
- modeling: The clean data structure facilitates the use of linear or generalized linear models to analyze clinical outcomes.

Example of visualization with `ggplot2`. For instance, you can visualize the distribution of PSI scores across your sample to assess the periodontal screening results at a glance:

```
# Standard visualization of PSI scores
ggplot(result_psi, aes(x = factor(psi), fill = factor(psi))) +
  geom_bar(show.legend = FALSE) +
  scale_fill_brewer(palette = "Reds") +
  theme_minimal() +
  labs(
    title = "Distribution of PSI Scores",
    x = "PSI Code",
    y = "Number of Patients"
  )
```



Example of correlation analysis. You can easily combine the index results with patient metadata. Here, we analyze the correlation between PSI scores and patient age.

```
# We add age to our results
patient_meta <- tibble(
  patient_id = 1:6,
  age = c(25, 42, 38, 55, 48, 30)
)

analysis_df <- result_psi %>%
  left_join(patient_meta, by = "patient_id")

# Spearman's Rank Correlation
# We use Spearman because PSI is an ordinal categorical scale
cor_test <- cor.test(analysis_df$psi, analysis_df$age, method = "spearman")

print(cor_test)

##
## Spearman's rank correlation rho
##
## data: analysis_df$psi and analysis_df$age
## S = 28.519, p-value = 0.7254
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## 0.185164
```

In this example, the test returns a Spearman's rho of 0.185 with a p-value of 0.725, indicating that there is no statistically significant correlation between age and PSI scores in this small sample. This workflow demonstrates how `tidy_dental()` outputs can be directly plugged into standard R hypothesis testing.

11. Conclusions

The `oralR` package provides a unified, robust, and extensible framework for standardizing and analyzing dental data in R.

It supports the workflow, moving beyond basic data manipulation to offer specialized tools for the dental field:

- **Advanced Notation Processing:** It detects and parses dental notations, handles ambiguous entries, and provides conversion between FDI, Universal, and alphanumeric systems.
- **Data Structuring:** The `tidy_dental()` function restructures dental records into tidy formats ready for statistical analysis.
- **Built-in Quality Control:** It ensures data integrity through internal checks, utilizing functions like `validate_index_input()` for periodontal scores and normalization routines for dmft/def data.
- **Validated Clinical Indices:** It automates the calculation of comprehensive caries, periodontal, and orthodontic indices (dmft, def, bop, pi, psi, pcr, icon).
- **Statistical Summaries:** It includes built-in functions such as `analyze_descriptives()` and `analyze_prevalence()` to easily extract descriptive metrics and prevalence rates from the processed datasets.
- **Integration with statistical workflows:** By returning standardized tibble objects, it allows users to pipe results into `ggplot2` for visualization and apply statistical models or tests without further data wrangling.

By bridging the gap between raw clinical records and validated statistical outputs, `ora1R` allows researchers and clinicians to focus on data analysis rather than data cleaning, generating reliable descriptive summaries and graphical outputs with ease.

`ora1R` supports the entire workflow from raw heterogeneous inputs to validated clinical indices, and comprehensive statistical summaries.